

Podrška za Kotlin u okviru LINVAST biblioteke

Opis sistema u okviru seminarskog rada
Matematički fakultet, Univerzitet u Beogradu

Andela Jovanović
mi251035@alas.matf.bg.ac.rs

Jun 2026.

Sažetak

Ovaj rad opisuje proširenje biblioteke LINVAST podrškom za programski jezik Kotlin. LINVAST je .NET biblioteka koja definiše jezički invarijantno apstraktno sintaksičko stablo (AST) za imperativne jezike, omogućavajući analizu i poređenje programa pisanih u različitim jezicima kroz jedinstvenu reprezentaciju.

Sadržaj

1	Opis problema	2
1.1	Motivacija	2
1.2	Cilj ovog rada	2
2	Arhitektura sistema	3
2.1	Struktura rešenja	3
2.2	Modul Kotlin builder-a	3
2.3	Automatsko otkrivanje	4
2.4	Tok obrade	4
3	Opis rešenja	5
3.1	Obrazac Posetilac i obilazak CST-a	5
3.2	Izgradnja čvorova: primer deklaracije	6
3.3	Hijerarhija izraza i prosleđivanje	7
3.4	Testiranje	8
3.5	Deklaracije	8
3.6	Funkcije	8
3.7	Deklaracije klase i interfejsa	9
3.8	Naredbe kontrole toka	10
3.9	Izrazi	11
3.10	Ilustrativni primer	12
4	Budući rad	12

1 Opis problema

U ovom poglavlju se opisuju motivacija za uvođenje jezički invarijantne statičke analize i konkretni ciljevi koji se ovim radom ostvaruju.

1.1 Motivacija

Alati za statičku analizu programa tipično su namenjeni jednom konkretnom programskom jeziku. Kada je potrebno primeniti istu vrstu analize, kao što su detekcija mrtvog koda, izračunavanje konstantnih izraza ili provera ispravnosti kontrolnog toka, na programe pisane u različitim jezicima, razvojni timovi su prinuđeni da održavaju zasebne, jezički specifične alate, ili da značajno suzuju obuhvat same analize.

Apstraktno sintaksičko stablo (skr. AST) nastaje kao rezultat sintaksičke analize ulaznog programa i čini osnovu za semantičku analizu i kasniju fazu kompilacije [1]. Budući da je AST vezan za gramatiku konkretnog programskog jezika, programi pisani u različitim jezicima ne mogu se direktno porediti posmatranjem njihovih stabala, što je posebno ograničavajuće u kontekstu migracije na nove tehnologije ili provere ekvivalentnosti implementacija.

LINVAST (Language-INVariant AST) je .NET biblioteka koja rešava ovaj problem definisanjem *jedinstvenog apstraktnog sintaksičkog stabla* sposobnog da predstavi programe iz više imperativnih programskih jezika. Za svaki podržani jezik postoji poseban *graditelj* (engl. *builder*) koji parsira izvorni kod i preslikava ga u ovu zajedničku reprezentaciju. Jednom kada se program prevede u LINVAST oblik, konstrukti poput deklaracija, izraza, kontrolnog toka i funkcija mogu se upoređivati, transformisati ili analizirati jezički nezavisnih posetiocima (engl. *visitors*), bez ikakve svesti o izvornom programskom jeziku [1].

1.2 Cilj ovog rada

Cilj ovog seminarskog rada je **proširiti LINVAST podrškom za Kotlin**. Kotlin je moderan, statički tipiziran jezik koji se izvršava na JVM platformi; njegova gramatika obuhvata konstrukte koji ne postoje u jezicima koje LINVAST već podržava (C, Lua, Go, Java). Konkretno, proširenje zahteva:

1. Preuzimanje ANTLR 4 gramatike za Kotlin i integrisanje generisanog C# leksera i parsera u projekat.
2. Implementaciju klase `KotlinASTBuilder`, raspoređene u šest parcijalnih `.cs` datoteka, koja obilazi generisano stablo parsiranja i proizvodi LINVAST AST čvorove.
3. Verifikaciju ispravnosti kroz postojeću testnu infrastrukturu.

Implementacija pokriva: deklaracije `val/var` svojstava, deklaracije funkcija na nivou datoteke, direktive uvoza (`import`), osnovni kontrolni tok (`if/else, while, for`), `return` naredbe, kompletnu hijerarhiju aritmetičko-logičkih izraza, kao i deklaracije klasa i interfejsa sa primarnim konstruktorom, naslednim tipovima i telom klase.

Lambda izrazi, ekstenzivne funkcije, koroutine i napredni tipski konstrukti izvan su obuhvata i razmatraju se kao budući rad u poglavlju 4.

2 Arhitektura sistema

U ovom poglavlju opisuje se kako je LINVAST rešenje strukturisano i gde se Kotlin builder uklapa u tu celinu.

2.1 Struktura rešenja

LINVAST rešenje sastoji se od četiri .NET projekta.

Projekat	Sadržaj
LINVAST	Osnovna biblioteka: apstraktna klasa <code>ASTNode</code> , interfejsi <code>IASTBuilder<TParser></code> i <code>IAbstractASTBuilder</code> , atribut <code>ASTBuilderAttribute</code> za automatsko otkrivanje, tipovi izuzetaka i podrška za beleženje.
LINVAST.Imperative	Konkretni tipovi AST čvorova za imperativne jezike (<code>SourceNode</code> , <code>FuncNode</code> , <code>DeclStatNode</code> , <code>IfStatNode</code> , ...) i svi jezički specifični graditelji: C, Lua, Go, Java, Kotlin .
LINVAST.Imperative.Visitors	Višekratno upotrebljivi posetioci AST-a: <code>ExpressionEvaluator</code> (sažimanje konstantnih izraza), <code>ConstantExpressionEvaluator</code> , <code>SymbolicExpressionBuilder</code> .
LINVAST.Tests	Testna skripta zasnovana na NUnit 3, organizovana po direktorijumima za svaki jezik; svaka klasa testova nasleđuje zajedničku osnovnu klasu koja pruža pomoćne metode za proveravanja.

Tabela 1: Struktura projekata u LINVAST rešenju

2.2 Modul Kotlin builder-a

Kotlin proširenje nalazi se u direktorijumu `LINVAST.Imperative/Builders/Kotlin/` i prati isti raspored koji koristi svaki drugi jezički graditelj u projektu:

```
Builders/Kotlin/  
+-- ANTLR/  
| +-- KotlinLexer.g4          (lekser gramatika)  
| +-- KotlinParser.g4         (parser gramatika)  
| +-- KotlinLexer.cs          (generisani lekser)  
| +-- KotlinParser.cs         (generisani parser)  
| +-- KotlinParserVisitor.cs  (generisani interfejs posetioca)  
| +-- KotlinParserBaseVisitor.cs (generisana osnovna klasa)  
+-- KotlinASTBuilder.cs       (ulazna tacka, deklaracija klase)  
+-- KotlinASTBuilder.Declarations.cs  
+-- KotlinASTBuilder.Functions.cs  
+-- KotlinASTBuilder.Statements.cs  
+-- KotlinASTBuilder.Expressions.cs  
+-- KotlinASTBuilder.Types.cs
```

`KotlinASTBuilder` je `sealed partial` klasa koja nasleđuje ANTLR-generisanu klasu `KotlinParserBaseVisitor<ASTNode>` i implementira interfejs `IASTBuilder<KotlinParser>`. Šest izvornih datoteka kompiliraju se zajedno putem mehanizma C# *delimičnih klasa* (engl. *partial class*), pri čemu svaka pokriva zasebnu oblast odgovornosti.

2.3 Automatsko otkrivanje

`ImperativeASTFactory` koristi refleksiju za pronalaženje graditelja u vreme izvođenja. Svaka klasa anotirana sa `[ASTBuilder(".kt")]` automatski se bira kada se datoteka sa ekstenzijom `.kt` prosledi metodi `BuildFromFile`.

2.4 Tok obrade

Celokupan put od izvornog Kotlin koda do LINVAST stabla prolazi kroz pet uzastopnih koraka prikazanih ispod:

`.kt datoteka` $\rightarrow$ `CharStream` $\rightarrow$ `KotlinLexer` $\rightarrow$ `CommonTokenStream` $\rightarrow$ `KotlinParser` $\rightarrow$
`KotlinASTBuilder` $\rightarrow$ `ASTNode`

Metoda `CreateParser` realizuje prvu polovinu ovog puta:

Isečak koda 1: Kreiranje parsera iz izvornog koda

```
1 public KotlinParser CreateParser(string code)
2 {
3     ICharStream stream = CharStreams.fromstring(code);
4     var lexer          = new KotlinLexer(stream);
5     lexer.AddErrorListener(new ThrowExceptionHandler());
6
7     ITokenStream tokens = new CommonTokenStream(lexer);
8     var parser          = new KotlinParser(tokens);
9     parser.RemoveErrorListeners();
10    parser.AddErrorListener(new ThrowExceptionHandler());
11
12    return parser;
13 }
```

Ulazni tekst se najpre obmotava u `CharStream` kako bi ANTLR mogao da ga čita karakter po karakter. `KotlinLexer` zatim vrši tokenizaciju, to jest grupiše karaktere u tokene poput ključnih reči (`val`, `fun`), operatora (`+`, `<`), literala i identifikatora. `CommonTokenStream` baferuje token niz kako bi parser mogao gledati unapred. `KotlinParser` čita niz tokena i gradi konkretno sintaksičko stablo (CST) u skladu sa pravilima Kotlin gramatike. Prilagođeni `ThrowExceptionHandler` na oba mesta zamenjuje ANTLR-ov podrazumevani listener koji štampa greške na konzoli - svaka sintaksička ili leksička greška se odmah pretvara u izuzetak.

Drugu polovinu puta realizuje metoda `BuildFromSource`:

Isečak koda 2: Izgradnja LINVAST stabla iz izvornog koda

```
1 public ASTNode BuildFromSource(string code)
2 {
3     return this.Visit(this.CreateParser(code).kotlinFile());
4 }
```

Poziv `.kotlinFile()` nalaže parseru da krene od korenskog gramatičkog pravila koje pokriva čitavu datoteku. `this.Visit(...)` zatim pokreće obilazak dobijenog CST-a pomoću metoda posetioca definisanih u `KotlinASTBuilder`-u i njegovim delimičnim datotekama, i vraća koren LINVAST stabla.

3 Opis rešenja

U ovom poglavlju detaljno se opisuju pristup implementaciji, preslikavanje pravila Kotlin gramatike u LINVAST čvorove i ključne odluke donete tokom razvoja.

3.1 Obrazac Posetilac i obilazak CST-a

ANTLR 4 generiše klasu `KotlinParserBaseVisitor<T>` koja implementira **obrazac Posetilac** (engl. *Visitor pattern*) nad konkretnim sintaksičkim stablom: za svako pravilo gramatike postoji odgovarajuća `Visit*` metoda. Svaka metoda koja nije redefinisana baca izuzetak čime se garantuje da svaki neimplementirani konstrukt odmah prijavljuje grešku umesto da prouzrokuje tiho pogrešno ponašanje. `KotlinASTBuilder` redefiniše isključivo metode konstrukata obuhvaćeni u implementacije.

Svaki posetilac prima kontekstni objekat `ctx` koji odražava tačno ono što je parser pronašao za dato pravilo: ako gramatičko pravilo glasi `block : LCURL statements RCURL`, tada `ctx` ima metode `.statements()` i pristupnike za leve i desne vitičaste zagrade. Obilazak podstabala se vrši rekurzivnim pozivom `this.Visit(ctx.subRule())`.

Odabir i prosleđivanje na višem nivou. Kotlin gramatika grupiše sve konstrukte na nivou datoteke pod pravilo `topLevelObject` (funkcije, svojstva, klase, sinonimi tipova). Posetilac proverava koji alternativni podizraz nije `null` i prosleđuje ga odgovarajućem posetiaocu:

Isečak koda 3: Odabir vrste konstrukta na nivou datoteke

```
1 public override ASTNode VisitTopLevelObject(  
2     KotlinParser.TopLevelObjectContext ctx)  
3 {  
4     if (ctx.functionDeclaration() != null)  
5         return this.Visit(ctx.functionDeclaration());  
6     if (ctx.propertyDeclaration() != null)  
7         return this.Visit(ctx.propertyDeclaration());  
8     // ... ostale alternative ...  
9     throw new NotImplementedException(  
10         $"Unsupported top-level object: {ctx.GetText()}");  
11 }
```

Isti obrazac primenjuje se i za naredbe unutar bloka - posetilac `VisitStatement` razlikuje deklaracije (`val x = 1`) od izraza koji se koriste kao naredbe (`x += 1`, pozivi funkcija).

Prenos greške kao izuzetka. Metoda `Visit` je redefinisana da uhvati `NullReferenceException` koji nastaje kada se pristupi opcionalnom gramatičkom elementu koji nije prisutan u konkretnom ulazu, i pretvori ga u `SyntaxErrorException` sa čitljivom porukom. Ovo olakšava dijagnozu problema bez otkrivanja internih detalja ANTLR obrade.

3.2 Izgradnja čvorova: primer deklaracije

Sledeći isečak prikazuje kako se gradi LINVAST čvor za deklaraciju `val/var` u pet koraka:

Isečak koda 4: Posetilac za deklaraciju svojstva

```
1 public override ASTNode VisitPropertyDeclaration(  
2     KotlinParser.PropertyDeclarationContext ctx)  
3 {  
4     // 1. keyword: "val" or "var"  
5     string keyword = ctx.VAL() != null ? "val" : "var";  
6  
7     // 2. variable name  
8     string name = ctx.variableDeclaration()  
9         .simpleIdentifier().GetText();  
10  
11     // 3. type name (null if no annotation)  
12     string? typeName = ctx.variableDeclaration()  
13         .type()?.GetText();  
14  
15     // 4. initializer (null if no assignment)  
16     ExprNode? init = ctx.expression() is not null  
17         ? this.Visit(ctx.expression()).As<ExprNode>()  
18         : null;  
19  
20     // 5. assemble LINVAST nodes  
21     var specs = typeName is not null  
22         ? new DeclSpecsNode(ctx.Start.Line, keyword, typeName)  
23         : new DeclSpecsNode(ctx.Start.Line, keyword);  
24     var decl = new VarDeclNode(ctx.Start.Line,  
25         new IdNode(ctx.Start.Line, name), init);  
26     return new DeclStatNode(ctx.Start.Line, specs,  
27         new DeclListNode(ctx.Start.Line, decl));  
28 }
```

Korak 1 : ključna reč.

`ctx.VAL()` i `ctx.VAR()` vraćaju token ako je odgovarajuća ključna reč prisutna u ulazu, ili `null` ako nije. Proverom koja od njih nije `null` dobija se niska "val" ili "var" koja se čuva za izgradnju `DeclSpecsNode`.

Korak 2 : ime promenljive.

Gramatičko pravilo `variableDeclaration` sadrži `simpleIdentifier` koji nosi ime promenljive. Pozivom `GetText()` dobija se niska, na primer "x" za deklaraciju `val x: Int`.

Korak 3 : tip (opciono).

Anotacija tipa (`: Int`) je opcionalna u Kotlinu i ako je izostavljena, `ctx.variableDeclaration().type()` vraća `null`. Operator `?.` osigurava da se u tom slučaju `typeName` postavi na `null` umesto da dođe do izuzetka.

Korak 4 : inicijalizator (opciono).

Ako postoji deo = izraz, rekurzivnim pozivom `this.Visit` se izraz prevodi u `ExprNode`. Koristi se `is not null` umesto `!= null` jer `ASTNode` preopterećuje operator `==` pa obrazac `is not null` uvek proverava referencu i zaobilazi preopterećeni operator.

Korak 5 : sklapanje čvorova.

Ekstraktovane vrednosti se slažu odozdo nagore: `DeclSpecsNode` nosi ključnu reč i tip, `VarDeclNode` nosi ime i inicijalizator, `DeclListNode` grupiše deklaratore, a `DeclStatNode` objedinjuje specifikatore i listu u potpunu naredbu deklaracije. Ovaj obrazac izdvajanja iz `ctx` i rekurzivnog sklapanja čvorova ponavlja se u svakom posetiaocu graditelja.

3.3 Hijerarhija izraza i prosleđivanje

Kotlin gramatika definiše prioritet operatora implicitno kroz lanac pravila od najnižeg ka najvišem prioritetu:

```
expression → disjunction (||) → conjunction (&&) → equalityComparison (==, !=) →  
comparison (<, >, ...) → additiveExpression (+, -) → multiplicativeExpression (*, /, %) →  
prefixUnaryExpression (-, !) → atomicExpression → literalConstant | simpleIdentifier
```

Svaki nivo posetioca prati isti obrazac: ako postoji samo jedan potomak (nema operatora na tom nivou), rezultat se odmah prosleđuje nižem posetiocu bez kreiranja novog čvora. Ako postoji više potomaka, gradi se binarni LINVAST čvor sa operatorom i levim i desnim operandom. Na primer:

Isečak koda 5: Posetilac za aditivni izraz

```
1 public override ASTNode VisitAdditiveExpression(  
2     KotlinParser.AdditiveExpressionContext ctx)  
3 {  
4     if (ctx.multiplicativeExpression().Length == 1)  
5         return this.Visit(ctx.multiplicativeExpression(0));  
6     int line = ctx.Start.Line;  
7     ExprNode left = this.Visit(ctx.multiplicativeExpression(0))  
8                     .As<ExprNode>();  
9     ExprNode right = this.Visit(ctx.multiplicativeExpression(1))  
10                    .As<ExprNode>();  
11     var op = ArithmOpNode.FromSymbol(line,  
12         ctx.additiveOperator(0).GetText());  
13     return new ArithmExprNode(line, left, op, right);  
14 }
```

Razmotrimo šta se dešava pri obradi izraza $3 + 4 * 2$. Parser gradi CST u kome `additiveExpression` ima dva potomka tipa `multiplicativeExpression`. Pošto levi potomak (3) nema operator, poziv `this.Visit` se rekurzivno spušta kroz lanac pravila sve do `VisitLiteralConstant`, koji proizvodi `LitExprNode(3)`. Desni potomak ($4 * 2$) ima dva potomka na nivou `multiplicativeExpression`, pa `VisitMultiplicativeExpression` gradi `ArithmExprNode("*", LitExprNode(4), LitExprNode(2))` na isti način. Konačni rezultat je:

```
ArithmExprNode ("+"  
+-- LitExprNode (3)  
+-- ArithmExprNode ("*"  
    +-- LitExprNode (4)  
    +-- LitExprNode (2)
```

Prioritet množenja nad sabiranjem je automatski sačuvan strukturom stabla.

3.4 Testiranje

Ispravnost svake grupe metoda posetioca proverava se kroz NUnit 3 testove koji su organizovani paralelno sa izvornim datotekama graditelja: `DeclarationTests`, `FunctionTests`, `StatementTests`, `ExpressionTests`, `ClassTests`, `TypeTests`. Svaki test kreira `KotlinASTBuilder`, poziva `BuildFromSource` sa konkretnim izvornim kodom i proverava strukturu dobijenog stabla. Testovi koriste specijalizovano polazno pravilo putem preopterećene metode koja prima `Func<KotlinParser, ParserRuleContext>`, što omogućava parsiranje samo jednog izraza ili naredbe bez potrebe za čitavom datotekom:

Isečak koda 6: Primer unit testa za deklaraciju

```
1 [Test]
2 public void ValTypedDeclarationTest()
3 {
4     DeclStatNode node = this.GenerateAST("val x: Int")
5                           .As<DeclStatNode>();
6     Assert.That(node.Specifiers.TypeName, Is.EqualTo("Int"));
7     Assert.That(node.DeclaratorList.Declarators.Single().Identifier,
8                 Is.EqualTo("x"));
9 }
```

3.5 Deklaracije

Izvorni kod: *KotlinASTBuilder.Declarations.cs*

Direktive uvoza. Gramatičko pravilo `importHeader` pokriva sve oblike uvoza. Puna putanja se čita metodom `ctx.identifier().GetText()` i čuva u `ImportNode`. Džoker (`import kotlin.math.*`) i pseudonim (`import Foo as Bar`) se parsiraju bez greške, ali se tiho ignorišu.

val/var deklaracije. Pravilo `propertyDeclaration` je centralno za sve deklaracije svojstava. Ime promenljive izdvaja se kroz pod-pravilo `variableDeclaration` (`VisitVariableDeclaration`), koje čita `simpleIdentifier` i opcionalnu tipsku anotaciju i vraća `VarDeclNode`. Graditelj razlikuje tri moguća slučaja na osnovu prisustva tipske anotacije i inicijalizatora:

Ulaz	TypeName	Identifer	Inicijalizator
<code>val x: Int</code>	"Int"	"x"	null
<code>val x: Int = 42</code>	"Int"	"x"	<code>LitExprNode(42)</code>
<code>var y = 0</code>	"var"	"y"	<code>LitExprNode(0)</code>

Tabela 2: Ponašanje `VisitPropertyDeclaration` za različite oblike ulaza

Kada tipska anotacija izostaje (treći red), `typeName` je null i `DeclSpecsNode` se konstruiše samo sa ključnom reči, pa je `Specifiers.TypeName` jednako "var". Ovo je privremeno rešenje jer inferencija tipa nije implementirana.

3.6 Funkcije

Izvorni kod: *KotlinASTBuilder.Functions.cs*

Pravilo gramatike `functionDeclaration` pokriva deklaracije funkcija na nivou datoteke. Graditelj izvlači:

- **ime** - iz `ctx.identifier().GetText()`;
- **povratni tip** - iz `ctx.type(0).GetText()` ukoliko je prisutna dvotačka iza zagrada; u suprotnom se podrazumeva "Unit";

- **parametre** - obilaskom `ctx.functionValueParameters()`, što daje `FuncParamsNode` sa listom `FuncParamNode` objekata; svaki parametar se gradi iz parova (tip, ime);
- **telo** - obilaskom `ctx.functionBody()` koji proizvodi `BlockStatNode`; ukoliko tela nema (apstraktna ili interfejs funkcija), prosledi se `null`.

Sintaksa sa izraznim telom (`fun double(x: Int) = x * 2`) nije podržana i baca `NotImplementedException`. Podrazumevane vrednosti parametara (`fun greet(name: String = "World")`) se parsiraju bez greške, ali podrazumevana vrednost se tiho odbacuje.

3.7 Deklaracije klasa i interfejsa

Izvorni kod: *KotlinASTBuilder.Declarations.cs*

Pravilo gramatike `classDeclaration` pokriva sve oblike deklaracija klasa i interfejsa. Graditelj proverava prisustvo ključnih reči `CLASS` i `INTERFACE` i na osnovu toga odlučuje koji `LINVAST` čvor da proizvede:

Isečak koda 7: Posetilac za deklaraciju klase

```

1 public override ASTNode VisitClassDeclaration(
2     KotlinParser.ClassDeclarationContext ctx)
3 {
4     int line = ctx.Start.Line;
5     var identifier = new IdNode(line, ctx.simpleIdentifier().GetText());
6
7     TypeNameListNode typeParams = ctx.typeParameters() != null
8         ? this.Visit(ctx.typeParameters()).As<TypeNameListNode>()
9         : new TypeNameListNode(line);
10
11     TypeNameListNode baseTypes = ctx.delegationSpecifiers() != null
12         ? this.Visit(ctx.delegationSpecifiers()).As<TypeNameListNode>()
13         : new TypeNameListNode(line);
14
15     var declarations = new List<DeclStatNode>();
16
17     if (ctx.primaryConstructor() != null) {
18         FuncParamsNode ctorParams =
19             this.Visit(ctx.primaryConstructor()).As<FuncParamsNode>();
20         FuncDeclNode ctorDecl = new FuncDeclNode(line, identifier,
21             ctorParams);
22         DeclSpecsNode ctorSpecs = new DeclSpecsNode(line, identifier.
23             Identifier);
24         declarations.Add(new DeclStatNode(line, ctorSpecs,
25             new DeclListNode(line, ctorDecl)));
26     }
27
28     if (ctx.classBody() != null)
29         declarations.AddRange(
30             this.Visit(ctx.classBody()).As<BlockStatNode>()
31                 .Children.Cast<DeclStatNode>());
32
33     var declSpecs = new DeclSpecsNode(line, identifier.Identifier);
34     var typeDecl = new TypeDeclNode(line, identifier,
35         typeParams, baseTypes, declarations);
36
37     if (ctx.INTERFACE() != null)
38         return new InterfaceNode(line, declSpecs, typeDecl);
39     return new ClassNode(line, declSpecs, typeDecl);
40 }

```

Primarni konstruktor. Kotlin dopušta da se konstruktor definiše direktno u zaglavlju klase:

```
class Person(val name: String, var age: Int).
```

Pravilo gramatike `primaryConstructor` sadrži `classParameters` to jest listu parametara u zagradama. Svaki parametar (`classParameter`) razlikuje se od parametra funkcije po tome što može imati prefiks `val/var` (čime se ujedno deklarira i svojstvo klase). Primarni konstruktor se preslikava u `FuncDeclNode` koji nosi ime klase kao identifikator i parametre kao `FuncParamsNode`, a zatim se uvija u `DeclStatNode` i dodaje na početak liste članova klase.

Nasledni tipovi. Lista naslednih tipova iza dvotačke (`class Dog : Animal`) čita se iz `delegationSpecifiers`. Svaki specifikator može biti običan tip (`userType`) ili poziv konstruktora (`constructorInvocation`). U oba slučaja izvlači se samo ime tipa i čuva u `TypeNameListNode`. Eksplicitna delegacija (`by` izraz) baca `NotImplementedException`.

Telo klase. `VisitClassBody` obilazi sve `classMemberDeclaration` i skuplja ih u `BlockStatNode`. `VisitClassMemberDeclaration` prosleđuje deklaracije funkcija i svojstava postojećim posetiocima (`VisitFunctionDeclaration`, `VisitPropertyDeclaration`), a za ugneždene klase poziva `VisitClassDeclaration` rekursivno. Ostali tipovi članova (sekundarni konstruktori, `init` blokovi, prateći objekti) bacaju `NotImplementedException`.

Ulaz	Čvor	Identifer	Napomena
<code>class Empty</code>	<code>ClassNode</code>	<code>Empty</code>	Bez konstruktora ni tela
<code>class Point(val x: Int)</code>	<code>ClassNode</code>	<code>Point</code>	Konstruktor kao prvi član
<code>class Dog : Animal</code>	<code>ClassNode</code>	<code>Dog</code>	<code>BaseTypes = ["Animal"]</code>
<code>interface Printable { }</code>	<code>InterfaceNode</code>	<code>Printable</code>	Prazno telo

Tabela 3: Ponašanje `VisitClassDeclaration` za različite oblike ulaza

3.8 Naredbe kontrole toka

Izvorni kod: *KotlinASTBuilder.Statements.cs*

if/else. Pravilo `ifExpression` može imati jednu ili dve grane tela tipa `controlStructureBody`. Graditelj proverava prisustvo drugog tela i gradi binarni ili ternarni `IfStatNode`:

- `if (x > 0) { return x } → IfStatNode(uslov, tada) (bez else);`
- `if (x > 0) { return x } else { return 0 } → IfStatNode(uslov, tada, inače).`

Telo grane može biti blok `{ ... }` (daje `BlockStatNode` putem `VisitBlock`) ili pojedinačan izraz bez vitičastih zagrada (daje `ExprStatNode`).

while. Pravilo `whileExpression` uvek daje `WhileStatNode` sa uslovom i telom. Ako telo nije prisutno, prosleđuje se prazan `BlockStatNode`.

for. Pravilo `forExpression` preslikava se u `ForeachStatNode`. Graditelj izdvaja promenljivu iteratora iz `variableDeclaration`, iterable iz `expression` i telo iz `controlStructureBody`. Podržana je i destruktivna iteracija (`multiVariableDeclaration`), gde se više promenljivih pakuje u jedan `DeclStatNode`.

return. Pravilo `jumpExpression` obuhvata `return`, `break`, `continue` i `throw`. Implementirani su samo oblici naredbe `return`:

- `return 42 → JumpStatNode(Return, LitExprNode(42));`
- `return → JumpStatNode(Return) bez izraza.`

Naredbe `break`, `continue` i `throw` bacaju `NotImplementedException`. Petlja `do-while` takode nije implementirana.

Dodela vrednosti. Pravilo `expression` pokriva i naredbe dodele (`x = 1`, `x += 1` itd.). Graditelj proverava prisustvo operatora dodele i gradi `ExprStatNode` koji sadrži `AssignExprNode` sa levim operandom, operatorom i desnim operandom.

3.9 Izrazi

Izvorni kod: *KotlinASTBuilder.Expressions.cs*, *KotlinASTBuilder.Types.cs*

Svi aritmetički, logički i relacioni izrazi grade se obilaskom hijerarhije pravila opisane u odeljku 3.3. Podrška po tipu izraza je sledeća:

- **Literali** (`literalConstant`): celi brojevi, decimalni brojevi, logičke vrednosti (`true/false`) i `null` se preslikavaju u `LitExprNode` odnosno `NullLitExprNode`. Decimalni brojevi se parsiraju sa `CultureInfo.InvariantCulture` kako bi separator uvek bila tačka, bez obzira na lokalna podešavanja operativnog sistema.
- **Identifikatori**: svaki `simpleIdentifier` daje `IdNode`. Kotlin dozvoljava meke ključne reči (`get`, `set`, `field` itd.) kao identifikatore, a budući da `VisitSimpleIdentifier` čita tekst tokena putem `ctx.GetText()`, ovi slučajevi se obrađuju automatski.
- **Aritmetika**: `+`, `-`, `*`, `/`, `%` daju `ArithmExprNode`.
- **Logika**: `||`, `&&` daju `LogicExprNode`; `!` (negacija) daje `UnaryExprNode`.
- **Poređenje**: `<`, `>`, `<=`, `>=`, `==`, `!=` daju `RelExprNode`.
- **Zagrade**: (izraz) zagrade se skidaju a unutrašnji izraz se obilazi; u stablu ne postoji zaseban čvor za zagrade.

Rezolucija tipova (*KotlinASTBuilder.Types.cs*). Svi tipski nazivi koji se pojavljuju u deklaracijama, parametrima funkcija i parametrima klasa prolaze kroz četiri posetioca iz ove datoteke:

- `VisitUserType` - spaja segmente kvalifikovanog naziva tačkama i proizvodi `TypeNameNode`, npr. `kotlin.io.File`;
- `VisitSimpleUserType` - čita jednostavan naziv tipa i ignoriše generičke argumente (`List<String>` → `TypeNameNode("List")`);
- `VisitNullableType` - dodaje `?` na kraj naziva tipa, npr. `String?` → `TypeNameNode(String?)`;
- `VisitTypeReference` - prosleđuje na `VisitUserType` ili rekursivno skida zagrade; baca izuzetak samo za `dynamic`.

Tip funkcije `((Int) -> String)` baca `NotImplementedException`.

3.10 Ilustrativni primer

Razmotrimo sledeću Kotlin funkciju koja izračunava apsolutnu vrednost celog broja:

Isečak koda 8: Primer Kotlin funkcije – apsolutna vrednost

```
1 fun absolute(x: Int): Int {  
2     if (x < 0) {  
3         return -x  
4     }  
5     return x  
6 }
```

Kada se ovaj isečak prosledi metodi `BuildFromFile`, `KotlinASTBuilder` obilazi stablo parsiranja i gradi sledeću LINVAST reprezentaciju:

SourceNode

```
+-- FuncNode (povratni tip: "Int", ime: "absolute")  
  +-- FuncParamsNode  
    | +-- FuncParamNode  
    |   +-- DeclSpecsNode ("Int")  
    |   +-- VarDeclNode ("x")  
  +-- BlockStatNode  
    +-- IfStatNode  
      | +-- RelExprNode ("<")  
      |   | +-- IdNode ("x")  
      |   | +-- LitExprNode (0)  
      |   +-- BlockStatNode  
      |     +-- JumpStatNode (Return)  
      |     +-- UnaryExprNode ("-")  
      |     +-- IdNode ("x")  
    +-- JumpStatNode (Return)  
    +-- IdNode ("x")
```

Primerimo da LINVAST stablo ne sadrži informacije o konkretnoj sintaksi Kotlina (zagrade, dvotačke, ključne reči) - sačuvana je isključivo semantička struktura koda. Ista analiza primenjena na semantički ekvivalentnu C funkciju proizvela bi strukturno identično stablo [1], što demonstrira suštinski cilj biblioteke.

4 Budući rad

Naredni koraci u razvoju Kotlin builder-a obuhvataju kako uklanjanje postojećih ograničenja, tako i proširenje obuhvata na konstrukte koji trenutno bacaju `NotImplementedException`:

- **Lambda izrazi i funkcije višeg reda** - Kotlinovi funkcionalni konstrukti zahtevaju proširenje ili aproksimaciju LINVAST-a, koji trenutno ne poseduje odgovarajući čvor.
- **Opsezi** - podrška za operator `..` (1..10 trenutno baca `NotImplementedException`).
- **Operatori bezbednosti od null vrednosti** - `?.`, `?:` (Elvis operator) i `!!`.
- **Pozivi metoda i pristup poljima** - postfiksne operacije poput `obj.method()` i `obj.field`.
- **Inferencija tipa** - automatsko određivanje tipa kada tipska anotacija izostaje (`var y = 0` treba da dobije `TypeName = "Int"`, a ne `"var"`).
- **Sekundarni konstruktori, init blokovi i prateći objekti** - konstrukti unutar tela klase koji trenutno bacaju izuzetak.

Literatura

- [1] I. Ristović, *Jezički invarijantna provera semantičke ekvivalentnosti strukturno sličnih segmenata imperativnog koda*, master rad, Matematički fakultet, Univerzitet u Beogradu, 2020.